"QA Analytics: Understanding Defect Trends and Predicting Test Execution Time"

🎯 Objective

To analyze QA defect and test execution logs to:

1. Explore key trends in defect data (EDA + visualizations).
2. Identify relationships between defect attributes (severity, module, tester, automation status, etc.).
3. Predict the **average test execution time** of a test case using regression analysis.

🗂️ Mock Dataset — Columns and Description

Column Name	Type	Description
`Defect_ID`	Categorical (string)	Unique defect identifier (e.g., D001, D002, …)
`Module_Name`	Categorical	Module where the defect occurred (Login, Payment, Dashboard, Reports…)
`Severity`	Categorical	Defect severity (Low, Medium, High, Critical)
`Priority`	Categorical	Defect priority (P3, P2, P1)
`Execution_Time`	Numeric (float)	Time (in minutes) to execute the related test case
`Defect_Fix_Time`	Numeric (float)	Time (in hours) taken to fix the defect
`Detected_Phase`	Categorical	Phase when defect was found (Unit Test, System Test, UAT)
`Automation_Covered`	Binary (0/1)	Whether the test is automated (1) or manual (0)
`Tester_Experience`	Numeric	Experience of tester in years
`Lines_of_Code`	Numeric	LOC impacted by defect (proxy for complexity)
`Defect_Reopened`	Binary (0/1)	Whether the defect was reopened
`Build_Release`	Categorical	Build number or release identifier

➡️ **Rows:** 200–300
➡️ You'll generate it using `numpy` and `pandas` (so you can control randomness).

🧮 Step-by-Step Workflow

1. Data Generation

- Use `numpy` and `pandas` to create random data for the above columns.

- Save it as `qa_defect_log.csv`.

2. Data Exploration

- Use `.info()`, `.describe()`, `.isnull().sum()` to understand missing values.
- Intentionally introduce a few missing values for practice.

3. Handling Missing Values

- Use `.fillna()` or `.dropna()` appropriately.
- For numeric: fill with median or mean.
- For categorical: fill with mode.

4. Outlier Detection

- Detect outliers in numeric columns (`Execution_Time`, `Defect_Fix_Time`, `Lines_of_Code`) using:
  - Z-score method
  - IQR method
- Remove or cap the outliers.

5. Feature Engineering

- Convert categorical columns using `pd.get_dummies()` (or `LabelEncoder`).
- Normalize numeric columns using `StandardScaler`, `MinMaxScaler`, and `RobustScaler` (to compare effects).

6. EDA & Visualization

- Visualize key relationships:
  - `Severity` vs `Defect_Fix_Time` (bar plot)
  - `Automation_Covered` vs `Execution_Time` (box plot)
  - Correlation heatmap between numeric columns
  - Distribution plots (histogram + KDE) for `Execution_Time` and `Defect_Fix_Time`
  - Scatter plot for `Lines_of_Code` vs `Execution_Time`
- Use both `matplotlib` and `seaborn`.

7. Model Building (Linear Regression)

- Objective: **Predict Execution_Time** based on defect and test features.
- Train-test split (`80/20`).
- Fit a `LinearRegression()` model:
- `from sklearn.linear_model import LinearRegression`
- `model = LinearRegression()`
- `model.fit(X_train, y_train)`
- `y_pred = model.predict(X_test)`
- Evaluate using:
  - `r2_score`

- `mean_squared_error`
- `root_mean_squared_error`

8. Insights & Interpretation

- Identify which features most strongly affect execution time.
- Example insights:
  - "Execution time increases with lines of code."
  - "Automated tests show 20% lower average execution time."
  - "Severity High defects tend to have longer execution and fix times."

💡 1. Test Case Effectiveness Analysis

Goal: Analyze test case execution data to identify which factors lead to failures and predict the *likelihood of a test case failing*.

🔷 Dataset Columns

Column	Type	Description
TestCase_ID	Categorical	Unique test ID
Module	Categorical	Application module name
Tester_Experience	Numeric	Experience in years
Test_Type	Categorical	Smoke, Regression, Sanity, Performance
Execution_Time	Numeric	Test execution time in minutes
Automation_Status	Binary	1 = Automated, 0 = Manual
Test_Steps	Numeric	Number of steps in the test case
Assertions_Count	Numeric	Number of validations/assertions
Data_Complexity	Numeric	1–10 scale of data complexity
Result	Binary	1 = Passed, 0 = Failed

🔷 Techniques Used

- Handle missing values, outliers (IQR/Z-Score)
- Normalize numeric columns (StandardScaler)
- Visualize pass/fail distribution
- Correlation heatmap (what impacts failures most)
- **Logistic Regression** → predict failure probability

🔷 Insights

- "Manual regression cases with >15 steps fail more often."
- "Execution time and data complexity strongly correlate with failure likelihood."

💡 2. QA Team Productivity Tracker

Goal: Study how tester workload, defect severity, and automation coverage affect average closure time.

🔹 Dataset Columns

Column	Description
Tester_ID	Unique ID
Module	Module under test
Total_TestCases_Executed	Count
Defects_Found	Count
Defect_Severity_Avg	Numeric (1–5 scale)
Automation_Coverage	% of automated cases
Execution_Hours	Total time spent
Avg_Closure_Time	Target variable (hours)

🔹 Techniques Used

- EDA with correlation plots
- Group-by analysis by tester/module
- Boxplots for closure time across severities
- Linear Regression → predict Avg_Closure_Time
- Outlier handling and normalization to stabilize the model

🔹 Insights

- "Testers handling highly automated modules spend 30% less time per closure."
- "Closure time increases non-linearly beyond 50 test cases per week."

💡 3. Defect Leakage Prediction

Goal: Identify which factors in the testing phase contribute to **defects escaping to UAT/Production**.

🔹 Dataset Columns

Column	Description
Defect_ID	Unique ID
Detected_Phase	Unit/System/UAT
Severity	Low/Medium/High/Critical
Module	Module name

Column	Description
Root_Cause	Coding / Environment / Data / Requirement
Test_Case_Coverage	% coverage for that module
Automation_Coverage	% automated tests
Lead_Experience	Numeric (years)
Leakage	Binary (1 = Leaked to UAT, 0 = Caught Earlier)

🔷 Techniques Used

- Missing values → mode imputation
- Outliers → IQR
- Label Encoding + Normalization
- EDA → bar plots, heatmaps, pie charts
- Logistic Regression → predict leakage risk

🔷 Insights

- "Modules with <60% test coverage have 40% higher leakage probability."
- "Automation alone doesn't prevent leakage; experienced leads play a key role."

💡 4. Automation ROI Analyzer

Goal: Estimate how much time and cost savings automation provides over manual testing across multiple releases.

🔷 Dataset Columns

Column	Description
Release_ID	Build/Release name
Total_TestCases	Total test cases per release
Automated_Cases	Count of automated cases
Avg_Manual_Time	Avg time per manual case
Avg_Automation_Time	Avg time per automated case
Defects_Found	Count
Defects_Fixed	Count
Total_Execution_Time	Dependent variable (hours)
Cost	Dependent variable (USD)

🔷 Techniques Used

- Feature engineering → Automation_Ratio = Automated_Cases / Total_TestCases
- Correlation & regression → to estimate time or cost savings
- Visualization → Automation Ratio vs Time Saved

🔷 Insights

- "Every 10% increase in automation coverage reduces execution time by 8 hours."
- "Cost drops linearly until 70% automation, after which ROI flattens."

💡 5. Release Quality Predictor

Goal: Predict overall *release quality score* (based on post-release defects, test pass rate, and automation) using regression.

🔷 Dataset Columns

Column	Description
Release_ID	Release number
Test_Pass_Rate	% passed
Automation_Coverage	% automated
Defect_Density	defects per 1k LOC
Critical_Defects	Count
Code_Churn	Lines changed
QA_Effort_Hours	Total QA hours
Release_Quality_Score	Target variable (1–100)

🔷 Techniques Used

- Outlier detection and normalization
- Correlation analysis
- Multiple linear regression
- R^2 and RMSE for evaluation

🔷 Insights

- "Automation coverage and test pass rate are top predictors of release quality."
- "Code churn negatively impacts release quality."

🧠 Summary Table

Project	Regression Type	Focus Area	Key Insight
QA Analytics (original)	Linear	Execution Time	What factors affect test execution
Test Effectiveness	Logistic	Pass/Fail Prediction	Failure probability analysis
QA Productivity	Linear	Closure Time	Tester workload efficiency
Defect Leakage	Logistic	Escape Prediction	Root causes of missed defects
Automation ROI	Linear	Time & Cost	Automation benefit estimation
Release Quality	Linear	Quality Score	Predict overall release health